DATA STRUCTURES & ALGORITHMS

Unit 2: Searching and Sorting — Question Bank & Model Solutions

Subject Code: FE/SE Computer Engg **Total Questions:** 13 Solutions **Target Marks:** 6 Marks Each

Question 1

Marks: 6 | CO: 2

Explain Linear search and Binary search with example. State its time complexity and compare linear and binary search (time and space complexity).

SOLUTION

1. Linear Search:

Linear search sequentially checks each element of a list starting from the first element until a match is found or the end of the array is reached. It does not require elements to be sorted.

Example: Search for element **23** in array `[10, 50, 23, 80, 40]`.

- Index 0: 10 (≠ 23)
- Index 1: 50 (≠ 23)
- Index 2: 23 (Match found at index 2!)

2. Binary Search:

Binary search works on a **sorted array** by repeatedly dividing the search interval in half. Compare the search key with the middle element: if equal, search completes; if smaller, search the left half; if larger, search the right half.

Example: Search key **23** in sorted array `[10, 20, 23, 40, 50, 80]`.

- Low = 0, High = 5. Mid = (0+5)/2 = 2. Array[2] = 23. Key found at index 2!

3. Comparison Table:

Parameter	Linear Search	Binary Search
Prerequisite	Array can be sorted or unsorted.	Array MUST be sorted.
Best Time Complexity	O(1) (First element)	O(1) (Middle element)
Worst Time Complexity	O(n)	$O(\log_2 n)$
Average Time Complexity	O(n)	$O(\log_2 n)$
Auxiliary Space Complexity	O(1) (Iterative)	O(1) (Iterative) / O(log n) (Recursive call stack)
Access Type	Sequential access	Direct / Random access

Question 2

Marks: 6 | CO: 2

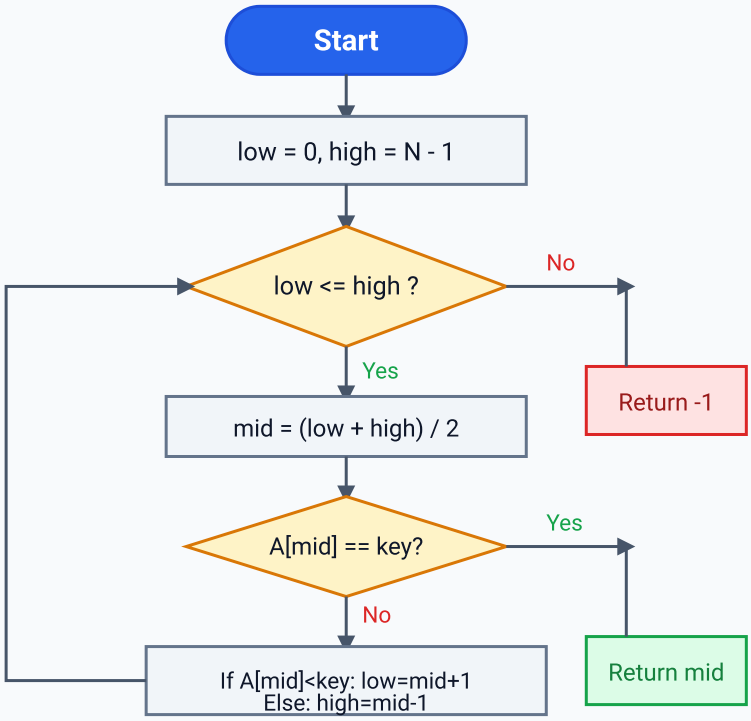
Write an algorithm for searching an element using binary search. Discuss the time complexity of algorithm in best case and worst case.

SOLUTION

Binary Search Algorithm (Iterative):

```
Algorithm BinarySearch(A, N, key): Input: Sorted array A of size N, target element key Output: Index of key if found, else -1 1. Set low = 0 2. Set high = N - 1 3. While low <= high do: a. Set mid = low + (high - low) / 2 b. If A[mid] == key then: Return mid // Key found c. Else if A[mid] < key then: Set low = mid + 1 // Search right half d. Else: Set high = mid - 1 // Search left half 4. Return -1 // Key not found in array
```

Binary Search Flowchart:



Time Complexity Analysis:

- **Best Case Time Complexity — O(1):** Occurs when the key is located exactly at the middle index during the very first step (e.g., `A[mid] == key`). Only 1 comparison is required.
- **Worst Case Time Complexity — O($\log_2$ n):** Occurs when the key is at either extreme end of the array or not present at all. In each iteration, the search space is halved: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \ldots \rightarrow 1$. Solving $n / 2^k = 1$ gives $k = log_2 n$ comparisons.

Question 3

Marks: 6 | CO: 2

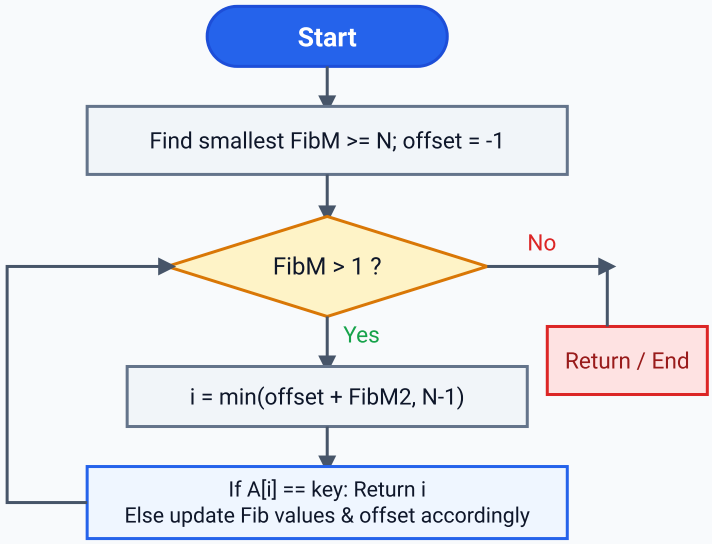
Write an algorithm for Fibonacci search and find out its time complexity.

SOLUTION

Fibonacci Search Concept: Fibonacci search is a comparison-based technique that uses Fibonacci numbers to search an element in a sorted array. It divides the array into unequal parts using Fibonacci numbers instead of exact halves.

```
Algorithm FibonacciSearch(A, N, key): 1. Find the smallest Fibonacci number
FibM >= N. Let FibM = Fib(k), FibM1 = Fib(k-1), FibM2 = Fib(k-2). 2. Set
offset = -1. 3. While FibM > 1 do: a. i = min(offset + FibM2, N - 1) b. If
A[i] < key then: FibM = FibM1; FibM1 = FibM2; FibM2 = FibM - FibM1; offset =
i c. Else if A[i] > key then: FibM = FibM2; FibM1 = FibM1 - FibM2; FibM2 =
FibM - FibM1 d. Else: Return i // Target found 4. If FibM1 == 1 and A[offset
+ 1] == key then return offset + 1 5. Return -1 // Target not found
```

Fibonacci Search Flowchart:



Time Complexity Analysis:

- **Worst-Case:** $O(\log_2 n)$. In each step, the algorithm reduces search range proportionally according to Fibonacci sequence ratios.
- **Best-Case:** O(1) when the target element matches the first probe index.
- **Space Complexity:** O(1) auxiliary space.

Question 4

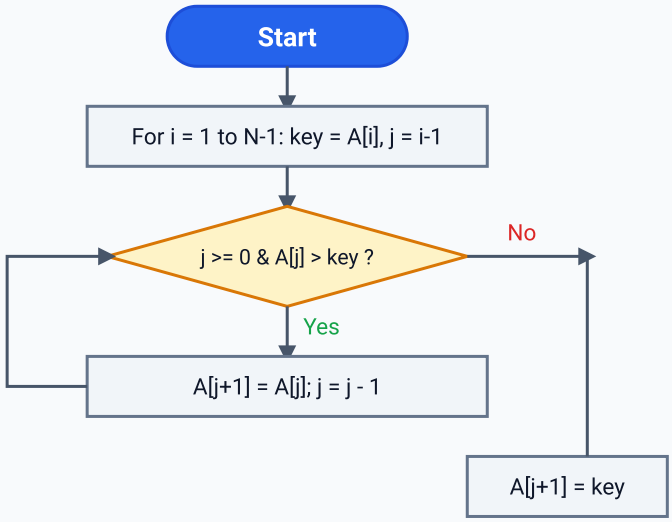
Marks: 6 | CO: 2

Explain insertion sort algorithm and sort the given list using insertion sort:
1) List: 7, 4, 10, 6, 3, 12, 1, 8, 2, 15, 9, 5
2) List: 55, 85, 45, 11, 34, 05, 89, 99, 67

SOLUTION

Insertion Sort Explanation: Insertion sort splits an array into sorted and unsorted regions. It picks elements sequentially from the unsorted region and inserts them into their correct position in the sorted region by shifting larger elements one position to the right.

Insertion Sort Flowchart:



Trace 1: Sort [7, 4, 10, 6, 3, 12, 1, 8, 2, 15, 9, 5]

- Initial Array: [7, 4, 10, 6, 3, 12, 1, 8, 2, 15, 9, 5]
- Pass 1 (Key=4): Shift 7 → [4, 7, 10, 6, 3, 12, 1, 8, 2, 15, 9, 5]
- Pass 2 (Key=10): No shift → [4, 7, 10, 6, 3, 12, 1, 8, 2, 15, 9, 5]
- Pass 3 (Key=6): Shift 10,7 → [4, 6, 7, 10, 3, 12, 1, 8, 2, 15, 9, 5]
- Pass 4 (Key=3): Shift 10,7,6,4 → [3, 4, 6, 7, 10, 12, 1, 8, 2, 15, 9, 5]
- Pass 5 (Key=12): No shift → [3, 4, 6, 7, 10, 12, 1, 8, 2, 15, 9, 5]
- Pass 6 (Key=1): Shift all → [1, 3, 4, 6, 7, 10, 12, 8, 2, 15, 9, 5]
- Pass 7 (Key=8): Shift 12,10 → [1, 3, 4, 6, 7, 8, 10, 12, 2, 15, 9, 5]
- Pass 8 (Key=2): Shift 12 down to 3 → [1, 2, 3, 4, 6, 7, 8, 10, 12, 15, 9, 5]
- Pass 9 (Key=15): No shift → [1, 2, 3, 4, 6, 7, 8, 10, 12, 15, 9, 5]
- Pass 10 (Key=9): Shift 15,12,10 → [1, 2, 3, 4, 6, 7, 8, 9, 10, 12, 15, 5]
- Pass 11 (Key=5): Shift 15..6 → [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15]

Trace 2: Sort [55, 85, 45, 11, 34, 5, 89, 99, 67]

- Initial: `[55, 85, 45, 11, 34, 5, 89, 99, 67]`
- Pass 1 (Key=85): `[55, 85, 45, 11, 34, 5, 89, 99, 67]`
- Pass 2 (Key=45): `[45, 55, 85, 11, 34, 5, 89, 99, 67]`
- Pass 3 (Key=11): `[11, 45, 55, 85, 34, 5, 89, 99, 67]`
- Pass 4 (Key=34): `[11, 34, 45, 55, 85, 5, 89, 99, 67]`
- Pass 5 (Key=5): `[5, 11, 34, 45, 55, 85, 89, 99, 67]`
- Pass 6 (Key=89): `[5, 11, 34, 45, 55, 85, 89, 99, 67]`
- Pass 7 (Key=99): `[5, 11, 34, 45, 55, 85, 89, 99, 67]`
- Pass 8 (Key=67): `[5, 11, 34, 45, 55, 67, 85, 89, 99]`

Question 5

Marks: 6 | CO: 2

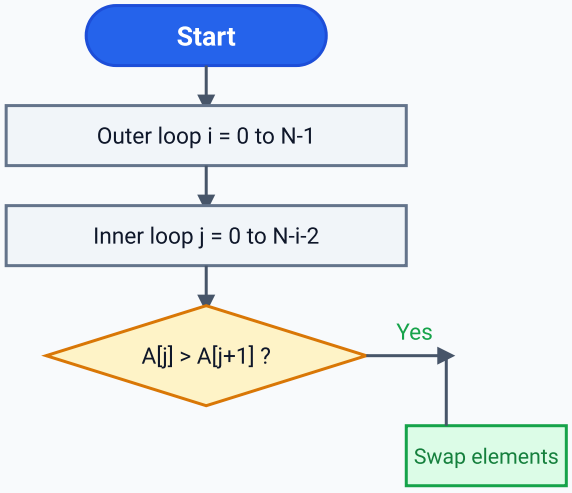
Write pseudo Python code for Bubble sort.

SOLUTION

Optimized Bubble Sort in Python:

```
def bubble_sort(arr): n = len(arr) # Traverse through all array elements for i in range(n): swapped = False # Last i elements are already in place for j in range(0, n - i - 1): # Traverse the array from 0 to n-i-1 # Swap if the element found is greater than the next element if arr[j] > arr[j + 1]: arr[j], arr[j + 1] = arr[j + 1], arr[j] swapped = True # If no two elements were swapped by inner loop, array is sorted if not swapped: break return arr # Example Usage data = [64, 34, 25, 12, 22, 11, 90] sorted_data = bubble_sort(data) print("Sorted Array:", sorted_data)
```

Bubble Sort Flowchart:



Question 6

Marks: 6 | CO: 2

Write a pseudo code for binary search apply your algorithm on the following no. stored in array to search no. 23 and 100.
Array: `9, 17, 23, 40, 45, 52, 58, 80, 85, 95, 100` **(Note: filled 00 as 100 based on standard array dataset)**

SOLUTION

Pseudocode:

```
function binarySearch(arr, key): low = 0 high = length(arr) - 1 while low <= high: mid = (low + high) // 2 if arr[mid] == key: return mid else if arr[mid] < key: low = mid + 1 else: high = mid - 1 return -1
```

Given Array (Indices 0 to 10): `[9, 17, 23, 40, 45, 52, 58, 80, 85, 95, 100]`

Trace 1: Search key = 23

- Step 1: low = 0, high = 10 → mid = (0+10)/2 = 5. `arr[5] = 52`. Since 23 < 52, set high = mid - 1 = 4.
- Step 2: low = 0, high = 4 → mid = (0+4)/2 = 2. `arr[2] = 23`. Match found at **index 2**!

Trace 2: Search key = 100

- Step 1: low = 0, high = 10 → mid = 5. `arr[5] = 52`. Since 100 > 52, set low = mid + 1 = 6.
- Step 2: low = 6, high = 10 → mid = (6+10)/2 = 8. `arr[8] = 85`. Since 100 > 85, set low = mid + 1 = 9.
- Step 3: low = 9, high = 10 → mid = (9+10)/2 = 9. `arr[9] = 95`. Since 100 > 95, set low = mid + 1 = 10.
- Step 4: low = 10, high = 10 → mid = 10. `arr[10] = 100`. Match found at **index 10**!

Question 7

Marks: 6 | CO: 2

Sort a given array using bubble sort method: `[64, 34, 25, 12, 22, 11, 90]`**. Show step by step execution for all passes highlighting swap/no-swap situations. How many passes are required?**

SOLUTION

Array Size N = 7. Total required passes = **N - 1 = 6 passes**.

Pass 1:

- (64, 34) → Swap → [34, 64, 25, 12, 22, 11, 90]
- (64, 25) → Swap → [34, 25, 64, 12, 22, 11, 90]
- (64, 12) → Swap → [34, 25, 12, 64, 22, 11, 90]
- (64, 22) → Swap → [34, 25, 12, 22, 64, 11, 90]
- (64, 11) → Swap → [34, 25, 12, 22, 11, 64, 90]
- (64, 90) → No Swap → [34, 25, 12, 22, 11, 64, 90]

Pass 2:

- (34, 25) → Swap, (34, 12) → Swap, (34, 22) → Swap, (34, 11) → Swap, (34, 64) → No Swap
- Result: `[25, 12, 22, 11, 34, 64, 90]`

Pass 3:

- (25, 12) → Swap, (25, 22) → Swap, (25, 11) → Swap, (25, 34) → No Swap
- Result: `[12, 22, 11, 25, 34, 64, 90]`

Pass 4:

- (12, 22) → No Swap, (22, 11) → Swap, (22, 25) → No Swap
- Result: `[12, 11, 22, 25, 34, 64, 90]`

Pass 5:

- (12, 11) → Swap, (12, 22) → No Swap
- Result: `[11, 12, 22, 25, 34, 64, 90]`

Pass 6:

- (11, 12) → No Swap. Array fully sorted!

Question 8

Marks: 6 | CO: 2

Enlist different searching methods. Write short note on:
i) Fibonacci search
ii) Index sequential search

SOLUTION

List of Searching Methods:

1. Linear / Sequential Search
2. Binary Search
3. Fibonacci Search
4. Index Sequential Search
5. Interpolation Search
6. Hashing (Hash table Search)

i) Fibonacci Search:

Fibonacci search is a divide-and-conquer search technique for sorted arrays using Fibonacci numbers. Unlike Binary Search which uses division by 2, Fibonacci search performs only addition and subtraction operations, making it useful on CPU architectures where division is computationally expensive. Time Complexity: O(log n).

ii) Index Sequential Search:

Index Sequential Search builds an auxiliary primary index table over a sorted main list. The index table stores key pointers at regular block intervals.
1. The index table is searched sequentially to identify the block where the key resides.
2. Linear search is performed only inside that specific block.
It offers faster access than pure linear search while being easier to update than standard binary search trees.

Question 9

Marks: 6 | CO: 2

Explain in brief the different searching techniques. What is the time complexity of each of them.

SOLUTION

Searching Technique	Brief Concept	Best Time	Worst Time
Linear Search	Checks each element sequentially from start to end. Works on unsorted data.	O(1)	O(n)
Binary Search	Repeatedly divides a sorted array in half by comparing with the middle element.	O(1)	O(log n)
Fibonacci Search	Uses Fibonacci numbers to partition sorted array ranges using addition/subtraction.	O(1)	O(log n)
Interpolation Search	Estimates key position using formula based on uniform distribution of sorted values.	O(1)	O(n) [Avg: O(log log n)]
Index Sequential Search	Uses a small index table to locate target block, then performs sequential search.	O(1)	O(√n + Block Size)

Question 10

Marks: 6 | CO: 2

Sort the following numbers step by step using insertion sort: `55, 85, 45, 11, 34, 5, 89, 99, 67`**. Comment on time complexity of Insertion sort.**

SOLUTION

Step-by-Step Sorting:

- **Initial Array:** `[55, 85, 45, 11, 34, 5, 89, 99, 67]`
- **Pass 1 (key = 85):** Compare with 55. 55 < 85, no shift → `[55, 85, 45, 11, 34, 5, 89, 99, 67]`
- **Pass 2 (key = 45):** Shift 85, 55 → `[45, 55, 85, 11, 34, 5, 89, 99, 67]`
- **Pass 3 (key = 11):** Shift 85, 55, 45 → `[11, 45, 55, 85, 34, 5, 89, 99, 67]`
- **Pass 4 (key = 34):** Shift 85, 55, 45 → `[11, 34, 45, 55, 85, 5, 89, 99, 67]`
- **Pass 5 (key = 5):** Shift all elements → `[5, 11, 34, 45, 55, 85, 89, 99, 67]`
- **Pass 6 (key = 89):** Compare with 85. No shift → `[5, 11, 34, 45, 55, 85, 89, 99, 67]`
- **Pass 7 (key = 99):** Compare with 89. No shift → `[5, 11, 34, 45, 55, 85, 89, 99, 67]`
- **Pass 8 (key = 67):** Shift 99, 89, 85 → `[5, 11, 34, 45, 55, 67, 85, 89, 99]`

Time Complexity Comments:

- **Best Case — O(n):** Occurs when the input array is already sorted. Only 1 comparison is performed per pass with zero shifts.
- **Worst Case — O(n^2):** Occurs when the array is sorted in reverse order. The inner loop performs $1 + 2 + ... + (n-1) = n(n-1)/2$ shifts.
- **Average Case — O(n^2):** Occurs when elements are randomly ordered.

Question 11

Marks: 6 | CO: 2

Explain in brief any three searching techniques. What is the time complexity of these techniques?

SOLUTION

1. Linear Search: Checks each element sequentially until key is found or end reached.
Complexity: Best = O(1), Worst = O(n), Average = O(n).

2. Binary Search: Divides a sorted array into two halves recursively by checking the midpoint.
Complexity: Best = O(1), Worst = O(log n), Average = O(log n).

3. Fibonacci Search: Uses Fibonacci progression numbers to determine split locations in a sorted array without division.
Complexity: Best = O(1), Worst = O(log n), Average = O(log n).

Question 12

Marks: 6 | CO: 2

Write an algorithm to search an element in array using binary search technique. Show stepwise search of Key 10 in `1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11`**. Is there any pre-processing of data required before starting binary search or what is its precursor requirement?**

SOLUTION

Precursor Requirement / Pre-processing:
Yes, the critical prerequisite for Binary Search is that the **array MUST be sorted** in ascending or descending order prior to executing the search algorithm. If the input array is unsorted, it must be pre-processed using a sorting algorithm (e.g., QuickSort/MergeSort in O(n log n) time).

Trace for Key = 10 in Array: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]` (Indices 0 to 10)

- **Step 1:** low = 0, high = 10
  mid = (0 + 10) / 2 = 5 → `A[5] = 6`
  Since key (10) > 6, narrow to right half: low = mid + 1 = 6.
- **Step 2:** low = 6, high = 10
  mid = (6 + 10) / 2 = 8 → `A[8] = 9`
  Since key (10) > 9, narrow to right half: low = mid + 1 = 9.
- **Step 3:** low = 9, high = 10
  mid = (9 + 10) / 2 = 9 → `A[9] = 10`
  `A[9] == 10`. Key successfully found at **Index 9**!

Question 13

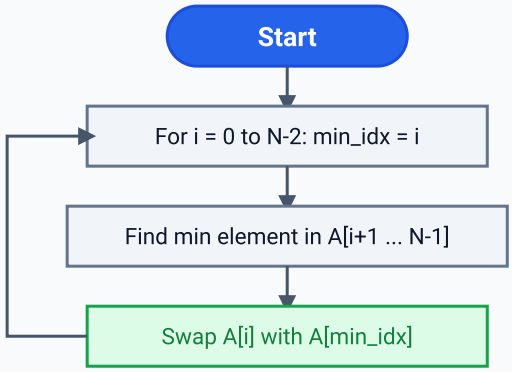
Marks: 6 | CO: 2

Sort given array by using selection sort method: `50, 23, 03, 18, 9, 01, 70, 21, 20, 6, 40, 04`**. Show step by step execution of all passes. What is the best and worst case time complexity of selection sort?**

SOLUTION

Selection Sort Execution: (Find minimum element in unsorted portion and swap with first element of unsorted portion)

Selection Sort Flowchart:



- Initial: `[50, 23, 3, 18, 9, 1, 70, 21, 20, 6, 40, 4]`
- **Pass 1:** Min is 1 (index 5). Swap with pos 0 (50) → `[1, 23, 3, 18, 9, 50, 70, 21, 20, 6, 40, 4]`
- **Pass 2:** Min from idx 1..11 is 3 (idx 2). Swap with pos 1 (23) → `[1, 3, 23, 18, 9, 50, 70, 21, 20, 6, 40, 4]`
- **Pass 3:** Min is 4 (idx 11). Swap with pos 2 (23) → `[1, 3, 4, 18, 9, 50, 70, 21, 20, 6, 40, 23]`
- **Pass 4:** Min is 6 (idx 9). Swap with pos 3 (18) → `[1, 3, 4, 6, 9, 50, 70, 21, 20, 18, 40, 23]`
- **Pass 5:** Min is 9 (idx 4). Already at pos 4 → `[1, 3, 4, 6, 9, 50, 70, 21, 20, 18, 40, 23]`
- **Pass 6:** Min is 18 (idx 9). Swap with pos 5 (50) → `[1, 3, 4, 6, 9, 18, 70, 21, 20, 50, 40, 23]`
- **Pass 7:** Min is 20 (idx 8). Swap with pos 6 (70) → `[1, 3, 4, 6, 9, 18, 20, 21, 70, 50, 40, 23]`
- **Pass 8:** Min is 21 (idx 7). Already at pos 7 → `[1, 3, 4, 6, 9, 18, 20, 21, 70, 50, 40, 23]`
- **Pass 9:** Min is 23 (idx 11). Swap with pos 8 (70) → `[1, 3, 4, 6, 9, 18, 20, 21, 23, 50, 40, 70]`
- **Pass 10:** Min is 40 (idx 10). Swap with pos 9 (50) → `[1, 3, 4, 6, 9, 18, 20, 21, 23, 40, 50, 70]`
- **Pass 11:** Min is 50 (idx 10). Already at pos 10 → `[1, 3, 4, 6, 9, 18, 20, 21, 23, 40, 50, 70]`

Selection Sort Time Complexity:

- **Best Case Time Complexity: $O(n^2)$** — Selection sort always performs $n(n-1)/2$ comparisons regardless of initial array order.
- **Worst Case Time Complexity: $O(n^2)$** — Same number of comparisons as best case.